DOCUMENTATION DU JEU — VOLUME 2 SUR 4

Architecture technique

Modules, classes, boucle de jeu et rendu pseudo-3D

Version 1.0.0 Auteur : Kossi Novembre 2026

1. Vue d'ensemble

Le projet est une application **libGDX 1.14.2** en Java 17, découpée en trois modules Gradle. Le code de jeu vit dans `core` et ne dépend d'aucune plateforme : le même binaire logique tourne sur bureau et sur Android.

MODULE	RÔLE	DÉPENDANCES
`core`	Logique, modèles, rendu, écrans, assets	libGDX (API seulement)
`lwjgl3`	Point d'entrée bureau (LWJGL 3.4.3), fenêtre 480 × 800	`core`, backend desktop
`android`	Activité Android, icône, natives par ABI	`core`, backend Android

Arborescence

```
Moto-Taxi-Rush/
├── assets/                 sprites, sons, musique (partagés)
├── core/src/main/java/cm/motorush/
│   ├── assets/             Assets.java
│   ├── audio/              AudioManager.java
│   ├── config/             GameConfig.java
│   ├── logic/              Road, Spawner, ScoreKeeper, DifficultyCurve, CollisionChecker
│   ├── model/              Moto, Obstacle, ObstacleType, Passager
│   ├── render/             Renderer, Perspective, MenuRenderer
│   ├── screen/             MenuScreen, PlayScreen, PauseScreen, GameOverScreen
│   └── storage/            HighScoreStore.java
├── core/src/test/java/     22 tests unitaires
├── android/                activité, manifeste, ressources
├── lwjgl3/                 lanceur bureau
├── docs/                   captures et présents PDF
└── tools/                  génération des assets
```

2. Séparation logique / rendu

La règle structurante du projet : **aucune classe de logique ne connaît le rendu**. `Moto`, `Spawner`, `Road`, `ScoreKeeper`, `DifficultyCurve` et `CollisionChecker` ne manipulent que des

nombres et des `Rectangle` (une classe mathématique de libGDX). Résultat : ces classes sont testables sans écran, sans OpenGL et sans assets, ce qui explique la présence de tests unitaires rapides sur la difficulté, le score et les collisions.

COUCHE	CLASSES	CONNAÎT
Modèle	`Moto`, `Obstacle`, `Passager`, `ObstacleType`	Rien d'autre que `GameConfig`
Logique	`Road`, `Spawner`, `ScoreKeeper`, `DifficultyCurve`, `CollisionChecker`	Modèle et configuration
Rendu	`Renderer`, `Perspective`, `MenuRenderer`	Modèle, logique, assets
Orchestration	`PlayScreen`, écrans de menu, `MotoRushGame`	Tout, mais reste mince

3. Rôle de chaque classe

CLASSE	RESPONSABILITÉ
`MotoRushGame`	Point d'entrée `Game` : crée les assets, l'audio, le stockage, instancie les écrans et gère les transitions (jouer, pause, fin, menu, quitter).
`PlayScreen`	Boucle de jeu : lit les entrées, avance la difficulté, la route, la moto, les apparitions et le score, gère collisions et ramassages, puis délègue le dessin au `Renderer`.
`AbstractMenuScreen`	Base commune des menus : navigation clavier et tactile (glissement vertical), validation, mise à jour des libellés.
`MenuRenderer`	Dessin d'un menu générique : fond, titre, sous-titre, options, signature, médaillon photo optionnel.
`Moto`	Voie courante, glissement interpolé, vies, invincibilité, clignotement, rectangle de collision.
`Spawner`	Création, descente et recyclage des obstacles et passagers, avec la règle des deux voies libres.
`Road`	Décalage des pointillés et distance totale parcourue (sert à caler les lampadaires).
`DifficultyCurve`	Vitesse, intervalle d'apparition et cycles d'heures de pointe en fonction du temps écoulé.
`ScoreKeeper`	Points de survie, points de course et combo.
`CollisionChecker`	Test rectangle contre rectangle, sans tolérance de bord.
`Perspective`	Projection pseudo-3D : profondeur, ordonnée écran, échelle, abscisse, suivi de caméra.
`Renderer`	Dessin complet de la scène : ciel, soleil, skyline, sol, route, lampadaires, sprites triés, particules, HUD.
`Assets`	Chargement unique des textures et sons via `AssetManager`, filtrage linéaire, libération en fin de vie.
`AudioManager`	Musique en boucle, bruitages de choc et de ramassage (hauteur montante), réglage persistant du son.
`HighScoreStore`	Meilleur score persistant via libGDX `Preferences`.
`GameConfig`	Toutes les constantes numériques et couleurs, organisées par étape.

4. Le rendu pseudo-3D

Aucune vraie caméra 3D n'est utilisée : tout est dessiné en 2D avec une projection manuelle en `1 / distance`. L'avantage est décisif — les voies, les collisions et le score restent calculés dans un plan 2D simple, donc les tests et l'équilibrage ne sont pas touchés par le rendu.

4.1 Principe

La caméra est posée derrière la moto, à hauteur fixe, et regarde vers un point de fuite situé sur la ligne d'horizon. Un point deux fois plus loin paraît deux fois plus petit et deux fois plus proche de l'horizon.

```
profondeur  d = y_monde − (BIKE_Y − CAMERA_BEHIND)
ordonnée    y_écran = HORIZON_Y − (HORIZON_Y − NEAR_Y) × NEAR_DEPTH / d
échelle     s = SCALE_NEAR × NEAR_DEPTH / d
abscisse    x_écran = WORLD_WIDTH / 2 + (x_monde − x_caméra) × s
```

Avec les valeurs du jeu : `HORIZON_Y = 470`, `NEAR_Y = 150`, `SCALE_NEAR = 1,2`, `CAMERA_BEHIND = 220`, `DRAW_DISTANCE = 1600`, `CAMERA_CULL_DEPTH = 40`.

4.2 Conséquences visuelles

- La moto tombe exactement sur `NEAR_Y` avec l'échelle 1,2 ; elle sert de référence stable à l'écran.
- L'horizon est une asymptote : rien ne le dépasse, ce qui évite les objets qui « flottent » au-dessus du ciel.
- Un objet dont la profondeur passe sous 40 unités est derrière la caméra : il n'est plus dessiné, ce qui évite les retournements d'échelle.
- La caméra suit la moto avec un retard (`CAMERA_FOLLOW_RATE = 7`) : lors d'un changement de voie, la moto se décale visiblement dans l'axe de la route au lieu de coller au centre de l'écran.

4.3 Ordre de dessin

La scène est découpée en couches, dessinées dans cet ordre :

1. Ciel en 24 bandes, du bleu nuit au rose de l'horizon.
2. Soleil et son halo (trois anneaux translucides).
3. Skyline : 16 immeubles de hauteur variable, avec fenêtres allumées et parallaxe horizontale de 0,12.
4. Sol de chaque côté de la route.
5. Route en trapèze, bandes de bord, pointillés des voies.
6. Lampadaires tous les 200 unités, avec halo lumineux.
7. Obstacles et passagers triés du plus lointain au plus proche.

8. Moto du joueur, toujours devant.
9. Particules, puis HUD en projection orthographique classique.

> Le tri des sprites est refait à chaque image : les objets sont triés par profondeur décroissante avant dessin, ce qui garantit qu'un camion lointain ne recouvre jamais un passager proche. Sans ce tri, deux sprites qui se chevauchent scintilleraient au passage de l'un devant l'autre.

4.4 Décor et effets

ÉLÉMENT	MISE EN ŒUVRE
Soleil couchant	Disque orange + trois anneaux de halo d'opacité croissante
Fenêtres éclairées	Grille de 3 × 2 cases par immeuble, motif déterministe (aucun clignotement)
Halos de lampadaires	Halo translucide autour de l'ampoule, opacité faible pour ne pas écraser la route
Tremblement d'écran	0,35 s après un choc, amplitude 12 unités, appliqué à la caméra de rendu
Particules	14 éclats projetés, gravité 700 u/s², durée de vie 0,45 s, convertis monde → écran

5. Boucle de jeu

Chaque image de `PlayScreen.render` suit toujours le même ordre, ce qui évite les dépendances cachées entre systèmes :

```
1. entrées (pause, changement de voie, tactile)
2. difficulté      → vitesse de route, intervalle, heure de pointe
3. moto            → glissement, invincibilité
4. caméra 3D       → recalage amorti sur la moto
5. route           → décalage des pointillés
6. apparitions     → descente, recyclage, créations
7. score           → points de survie
8. ramassages      → passagers touchés, combo, particules
9. collisions      → vies, invincibilité, secousse, son
10. fin de partie  → si plus de vie
11. dessin         → Renderer puis HUD
```

Point important : la caméra est mise à jour *avant* la route et les apparitions, mais *après* la moto. La projection utilise donc la position de caméra de l'image courante, sans décalage d'une image entre la moto et son environnement.

6. Persistance

Aucune base de données : libGDX `Preferences` fournit un petit magasin clé/valeur propre à chaque plateforme (fichier XML sur bureau, `SharedPreferences` sur Android).

FICHIER	CLÉ	TYPE	ÉCRITURE
`moto-taxi-rush`	`highScore`	entier	À la fin de chaque partie qui bat le record
`moto-taxi-rush`	`soundEnabled`	booléen	À chaque bascule depuis le menu ou la pause

7. Module Android

PARAMÈTRE	VALEUR
Identifiant	`cm.motorush.android`
Activité	`cm.motorush.android.AndroidLauncher`
SDK minimum / cible	24 (Android 7.0) / 36
Orientation	Portrait, plein écran sans barre de titre
ABI embarquées	`arm64-v8a`, `armeabi-v7a`, `x86_64`
Assets	Le dossier `assets/` du projet, sans copie

Les bibliothèques natives de libGDX sont livrées sous forme de jars : la tâche `copyAndroidNatives` les décompresse dans `android/libs/<abi>/` avant chaque build, et le plugin Android les empaquette dans l'APK.

8. Correspondance exigences / code

EXIGENCE	IMPLÉMENTATION
R1 — écran logique fixe	`WORLD_WIDTH/HEIGHT` + `FitViewport`
R2 — trois voies égales	`LANE_COUNT`, `LANE_WIDTH`, `Moto.laneCenterX`
R3 — moto et déplacement d'une voie	`Moto.changeLane`, interpolation sur 0,16 s
R4 — pas de sortie de route	`Moto.changeLane` ignore une cible hors bornes
R5 — voitures, camions, nids-de-poule	`Obstacle`, `ObstacleType`, `Spawner`
R6 — accélération de la route	`DifficultyCurve.getRoadSpeed`
R7 — apparitions plus fréquentes	`DifficultyCurve.getSpawnInterval`
R8 — une voie toujours libre	`Spawner.trySpawn` (règle des deux voies libres)
R9 — vies et invincibilité	`Moto.hit`, `CollisionChecker`
R10 — passagers	`Passager`, `Spawner.collectPassager`
R11 — passager raté sans pénalité	`Spawner.moveAndRemoveOffScreen`
R12 — score	`ScoreKeeper`
R13 — fin de partie et meilleur score	`PlayScreen`, `GameOverScreen`, `HighScoreStore`

9. Choix techniques assumés

- **Pas de vraie 3D.** Une projection manuelle suffit pour l'effet recherché et garde toute la logique en 2D, donc testable.
- **Pas d'`AssetManager` asynchrone.** Les assets sont chargés au démarrage en une fois : le jeu est petit, un écran de chargement serait inutile.
- **Pas d'allocation par image.** Les listes, tableaux et rectangles sont réutilisés ; le tri des sprites se fait sur une liste pré-existante.
- **Configuration centralisée.** `GameConfig` rassemble toutes les valeurs, y compris les couleurs, pour régler le jeu sans chercher dans le code.
- **Sprites générés.** Les images sont produites par script plutôt que dessinées à la main : le style reste homogène et reproductible.